

Lekcja 12

Temat: Obsługa plików: Otwieranie (with open), czytanie/zapis (read, write), tryby (r, w, a).

Obsługa plików

Python ma bardzo prosty i **bezpieczny sposób pracy z plikami** dzięki wbudowanej funkcji `open()` oraz menedżerowi kontekstu `with`. Dzięki temu nie musisz pamiętać o ręcznym zamykaniu pliku (`close()`), a kod jest czytelniejszy i mniej podatny na błędy.

1. Otwieranie pliku – with open(...)

Zalecany sposób:

```
with open('nazwa_pliku.txt', 'tryb') as plik:
    # tutaj pracujemy z plikiem
    # po wyjściu z bloku with plik jest automatycznie zamykany
```

with

- Automatycznie zamyka plik nawet jeśli wystąpi błąd (wyjątek).
- Kod jest krótszy i czytelniejszy.
- Nie musisz pamiętać o `plik.close()`.

2. Tryby otwierania plików (mode)

'r' – read (domyślny)

Otwiera plik tylko do odczytu. Plik **musi istnieć**, inaczej dostaniesz błąd.

'w' – write

Tworzy nowy plik lub **całkowicie nadpisuje** istniejący.

'a' – append

Dopisuje dane na końcu pliku. Jeśli plik nie istnieje – automatycznie go tworzy.

Dodatkowo często spotykane:

- **'r+'** – odczyt + zapis

- 'w+' – odczyt + zapis (nadpisuje)
- 'a+' – odczyt + dopisywanie

Przykład otwierania w różnych trybach:

```
with open('dane.txt', 'r') as plik:    # tylko odczyt
    ...

with open('wyniki.txt', 'w') as plik:  # nadpisanie
    ...

with open('log.txt', 'a') as plik:    # dopisywanie
    ...
```

3. Czytanie pliku – metody read(), readline(), readlines()

Przykład - czytanie całego pliku:

```
with open('dane.txt', 'r', encoding='utf-8') as plik:
    # 1. Odczyt całego pliku naraz (najprostsze)
    zawartosc = plik.read()
    print(zawartosc)

    # 2. Odczyt linia po linii (najlepsze dla dużych plików)
    for linia in plik:
        print(linia.strip())    # strip() usuwa \n na końcu

    # 3. Odczyt jako lista linii
    linie = plik.readlines()
    print(linie)
```

Praktyczne przykłady zastosowania:

```
# Przykład 1: Wczytanie konfiguracji
with open('config.txt', 'r', encoding='utf-8') as plik:
    for linia in plik:
        if '=' in linia:
            klucz, wartosc = [x.strip() for x in linia.split('=', 1)]
            print(f"{klucz} = {wartosc}")

# Przykład 2: Wczytanie listy zakupów do listy Pythona
with open('zakupy.txt', 'r', encoding='utf-8') as plik:
    lista_zakupow = [linia.strip() for linia in plik if linia.strip()]
    print(lista_zakupow)
```

4. Zapis pliku – metoda write() i writelines()

```
# Tryb 'w' → nadpisuje plik
with open('wyniki.txt', 'w', encoding='utf-8') as plik:
    plik.write("Wynik obliczeń: 42\n")
    plik.write("Druga linia tekstu\n")

# Można też zapisać wiele linii naraz
linie = ["Pierwsza\n", "Druga\n", "Trzecia\n"]
plik.writelines(linie)

# Tryb 'a' → dopisuje na końcu (idealny do logów)
import datetime

with open('log.txt', 'a', encoding='utf-8') as plik:
    czas = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    plik.write(f"[{czas}] Użytkownik załogował się\n")
    plik.write(f"[{czas}] Błąd: coś poszło nie tak\n")
```

Praktyczne przykłady zastosowania:

```
import datetime
# Przykład 1: Zapis listy zakupów (nadpisanie)
zakupy = ["mleko", "chleb", "masło", "jajka"]

with open('lista_zakupow.txt', 'w', encoding='utf-8') as plik:
    for produkt in zakupy:
        plik.write(produkt + '\n')

# Przykład 2: Logowanie błędów (dopisywanie)
try:
    1 / 0
except Exception as e:
    with open('errors.log', 'a', encoding='utf-8') as plik:
        plik.write(f"[{datetime.datetime.now()}] BŁĄD: {e}\n")
```

Lekcja 13 – TERMIN ODDANIA PROJEKTÓW

Temat: Brak modyfikatorów dostępu

W Pythonie nie ma prawdziwych modyfikatorów dostępu (jak *public*, *private*, *protected* w Javie/C++/C#). Nie istnieją żadne słowa kluczowe do tego celu.

Python używa tylko **konwencji nazewnictwa** (i jednego mechanizmu name mangling). Są dokładnie **trzy poziomy**:

Poziom	Zapis	Co oznacza	Czy naprawdę chroniony?
Public	<code>self.nazwa</code>	Domyślny – każdy może czytać i modyfikować	Nie – całkowicie otwarty
Protected	<code>self._nazwa</code>	Tylko konwencja („nie ruszaj z zewnątrz klasy”)	Nie – dostępny zawsze
Private	<code>self.__nazwa</code>	Name mangling + konwencja	Prawie tak (ale da się obejść)

- **Public** – bez podkreślnika
Wszystko, co nie ma _ na początku, jest publiczne.
- **Protected** – pojedynczy _
Tylko sygnalizuje innym programistom: „to jest wewnętrzne, lepiej nie używaj bezpośrednio”.
Python **nie blokuje** dostępu. W Pythonie **protected** to **tylko konwencja**, nie prawdziwa ochrona.
- **Private** – dwa _
Python automatycznie zmienia nazwę na `_NazwaKlasy_nazwa` (name mangling).
Dzięki temu nie da się przypadkowo odwołać z zewnątrz, ale **nadal można** dostać się przez `_Klasa_nazwa`. To jest **najmocniejsza forma "prywatności"** jaką Python oferuje. Python robi coś specjalnego nazywanego **name mangling** („przekształcanie nazwy”).

Przykład:

```
class Osoba:
    def __init__(self, imie, wiek):
        self.imie = imie          # ← PUBLICZNA (self.nazwa)
        self._wiek = wiek         # protected (tylko dla porównania)
        self.__haslo = "sekret"   # private (tylko dla porównania)

    def przedstaw_sie(self):
        print(f"Cześć, jestem {self.imie}")
```

```
class Raport:

    def generuj_raport(self, osoba: Osoba):

        print(f"RAPORT:")
        print(f"  Imię: {osoba.imie}")          # ← działa zawsze
```

```
# print(osoba._wiek)      # działa, ale nie powinieneś
# print(osoba.__haslo)    # AttributeError
```

```
class Pracownik(Osoba):
    def __init__(self, imie, wiek, stanowisko):
        super().__init__(imie, wiek)
        self.stanowisko = stanowisko

    def pokaz_dane(self):

        print(f"Pracownik: {self.imie}")
        print(f"Stanowisko: {self.stanowisko}")

jan = Osoba("Jan Kowalski", 35)
anna = Pracownik("Anna Nowak", 28, "programista")

raport = Raport()
raport.generuj_raport(jan)
raport.generuj_raport(anna)

anna.pokaz_dane()

print(f"Bezpośrednio: {jan.imie}")
```

Dlaczego nie stosować **Protected** – pojedynczy `_`

1. Wyobraź sobie, że klasa `Osoba` jest napisana przez kogoś innego (np. bibliotekę, kolegę z zespołu, albo Ciebie sprzed 6 miesięcy).

```
class Osoba:
    def __init__(self, imie, wiek):
        self.imie = imie
        self._wiek = wiek          # ← PROTECTED (tylko konwencja!)

    def urodziny(self):
        self._wiek += 1

    def pokaz_wiek(self):
        print(f"{self.imie} ma {self._wiek} lat")
```

2. Co się stanie, jeśli Ty użyjesz `osoba._wiek` z zewnątrz?

To działa... ale jest **niebezpieczne**.

Co się stało z kodem, który używał jan._wiek?

4. TAK POWINNO SIĘ ROBIĆ (najlepsza praktyka 2025)

```
class Osoba:
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # protected - tylko wewnątrz

    def urodziny(self):
        print(f"🎂 Wszystkiego najlepszego {self.imie}!")

# ===== PUBLICZNY INTERFEJS =====
@property
def wiek(self):
    # getter - tylko odczyt
```

```

        """Zwraca aktualny wiek"""
        today = date.today()
        wiek = today.year - self._data_urodzenia.year
        if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):
            wiek -= 1
        return wiek

    @wiek.setter
    def wiek(self, nowy_wiek):          # setter - bezpieczna zmiana
        if nowy_wiek < 0:
            raise ValueError("Wiek nie może być ujemny!")
        # przeliczamy datę urodzenia na podstawie nowego wieku
        today = date.today()
        self._data_urodzenia = date(today.year - nowy_wiek, today.month,
today.day)

jan = Osoba("Jan Kowalski", date(1995, 3, 15))

print(jan.wiek)          # 30 (lub aktualny wiek) ← OK
jan.wiek = 31            # zmienia wiek bezpiecznie ← OK
jan.urodziny()

# jan._data_urodzenia    # ← działa technicznie, ale NIE RÓB TEGO!

```

Przykład Private – dwa podkreślniki __

```

class KontoBankowe:
    def __init__(self, wlasciciel):
        self.wlasciciel = wlasciciel
        self.__saldo = 5000          # ← PRIVATE (dwa __)

    def wpłac(self, kwota):
        self.__saldo += kwota
        print(f"Wpłacono {kwota} zł")

    def wypłac(self, kwota):
        if kwota <= self.__saldo:    # używamy wewnątrz klasy – OK
            self.__saldo -= kwota
            print(f"Wypłacono {kwota} zł")
        else:
            print("Brak środków!")

    def pokaz_saldo(self):
        print(f"Saldo: {self.__saldo} zł")    # też wewnątrz klasy

```

```

konto = KontoBankowe("Jan Kowalski")

konto.wplac(1000)
konto.pokaz_saldo()          # działa → Saldo: 6000 zł

# Próba bezpośredniego dostępu (to, co większość ludzi próbuje):
print(konto.__saldo)        # ← AttributeError: 'KontoBankowe' object has no
                             attribute '__saldo'

# Próba zmiany:
# konto.__saldo = 999999    # też nie działa tak, jak myślisz

```

Jak obejść name mangling?

```

# Prawdziwa nazwa po manglingu:
print(konto._KontoBankowe__saldo)  # ← działa! → 6000

# Możemy nawet zmienić:
konto._KontoBankowe__saldo = 100   # teraz saldo = 100 zł
konto.pokaz_saldo()                 # → Saldo: 100 zł

```

Lekcja 14

Temat: Zastosowania modułów i pakietów: Import, from-import, tworzenie własnych modułów, init.py

Moduł to po prostu plik z rozszerzeniem .py, który zawiera kod Pythona (funkcje, klasy, zmienne, instrukcje).

Dzięki modułom możesz podzielić swój kod na mniejsze, logiczne części i używać go wielokrotnie w innych plikach.

1. Importowanie modułu – import

```

import math
import random
import os

```

```
print(math.pi)
print(math.sqrt(16))

liczba = random.randint(1, 10)
print(liczba)

print(f"Aktualny katalog roboczy: {os.getcwd()}")
```

2. Importowanie konkretnych rzeczy – from ... import ...

```
from math import sqrt, pi
from random import randint, choice
from os import getcwd

print(sqrt(25))
print(pi)

losowa = randint(1, 100)
print(losowa)

print(f"Aktualny katalog roboczy: {getcwd()}")
```

Możesz też importować wszystko (niezalecane w dużych projektach):

```
from math import *
```

3. Tworzenie własnych modułów

Przykład struktury:

```
moj_projekt/
├── main.py
├── obliczenia.py
└── utils.py
```

Plik obliczenia.py (moduł):

```
# obliczenia.py

def dodaj(a, b):
    return a + b
```

```
def odejmij(a, b):
    return a - b

def pomnoz(a, b):
    return a * b

# zmienne na poziomie modułu
PI = 3.14159
TAX_RATE = 0.23

class Kalkulator:
    def __init__(self):
        self.wynik = 0

    def dodaj(self, x):
        self.wynik += x
        return self.wynik
```

Plik main.py (używamy modułu):

```
import obliczenia
from obliczenia import dodaj, Kalkulator

print(obliczenia.dodaj(5, 7))           # 12
print(dodaj(10, 20))                   # 30

kalk = Kalkulator()
print(kalk.dodaj(100))                  # 100
print(kalk.dodaj(50))                   # 150
```

4. Pakiety (packages) – struktura z wieloma modułami

Pakiet to folder zawierający plik `__init__.py` (w Python 3.3+ nie jest już obowiązkowy, ale nadal bardzo zalecany).

Przykład struktury pakietu:

```
sklep/
├── __init__.py
├── produkty.py
├── zamowienia.py
└── platnosci/
```

```
├── __init__.py
├── karta.py
├── przelew.py
└── utils/
    ├── __init__.py
    └── helpers.py
```

Plik sklep/produkty.py:

```
# sklep/produkty.py

class Produkt:
    def __init__(self, nazwa, cena):
        self.nazwa = nazwa
        self.cena = cena

    def stworz_produkt(nazwa, cena):
        return Produkt(nazwa, cena)
```

Plik sklep/zamowienia.py:

```
# sklep/zamowienia.py

from .produkty import Produkt    # import względny

class Zamowienie:
    def __init__(self):
        self.produkty = []

    def dodaj_produkt(self, produkt):
        self.produkty.append(produkt)
```

Plik sklep/__init__.py

```
# sklep/__init__.py

# Możemy tutaj wystawić wygodny interfejs publiczny pakietu

from .produkty import Produkt, stworz_produkt
from .zamowienia import Zamowienie

# Możemy też wykonać kod inicjalizujący pakiet
print("Pakiet 'sklep' został zaimportowany")
```

```
__all__ = ['Produkt', 'stworz_produkt', 'Zamowienie']
```

Użycie pakietu w main.py:

```
# main.py

import sklep
from sklep import Produkt, stworz_produkt, Zamowienie

p1 = stworz_produkt("Laptop", 4999)
p2 = Produkt("Myszka", 89)

zam = Zamowienie()
zam.dodaj_produkt(p1)
zam.dodaj_produkt(p2)

print(f"Złożono zamówienie na {len(zam.produkty)} produktów")
```

6. Rola pliku __init__.py

Plik __init__.py ma kilka ważnych funkcji:

1. **Zamienia folder w pakiet** (Python wie, że to nie zwykły folder)
2. **Kod inicjalizujący** – wykonuje się przy pierwszym imporcie pakietu
3. **Definiowanie __all__** – kontroluje co jest importowane przy from pakiet import *
4. **Wystawianie API** – wygodne importy z poziomu pakietu

Przykład zaawansowanego __init__.py:

```
# sklep/__init__.py

from .produkty import Produkt, stworz_produkt
from .zamowienia import Zamowienie
from .platnosci.karta import PlatnoscKarta

# Możesz stworzyć aliasy
from .utils.helpers import formatuj_cene as fc

__version__ = "1.2.3"
__all__ = ['Produkt', 'Zamowienie', 'stworz_produkt', 'PlatnoscKarta']
```

Dzięki temu użytkownik pakietu może pisać:

```
from sklep import Produkt, Zamowienie
# zamiast:
# from sklep.produkty import Produkt
# from sklep.zamowienia import Zamowienie
```

